Homotopy type theory with explicit conversions

Benno van den Berg
ILLC, University of Amsterdam

Foundations and Applications of Univalent Mathematics
Herrsching, 20 December 2019

Thanks to . . .

- Martijn den Besten (Amsterdam)
- Menno de Boer (Stockholm)
- Peter Lumsdaine (Stockholm)

Section 1

Why explicit conversions?

A nice ideal

An important design decision for type theory is that $\Gamma \vdash a \in \sigma$ should be decidable (one can decide whether it is derivable or not).

Why do we want this?

A nice ideal

An important design decision for type theory is that $\Gamma \vdash a \in \sigma$ should be decidable (one can decide whether it is derivable or not).

Why do we want this?

The philosophical argument: under the proposition-as-types interpretation σ is a mathematical statement and a is a purported proof. Proofs are supposed to rationally compel assent: so if a is really a proof of σ, an (idealised) rational agent cannot fail to see this.

Clearly, we make several idealisations here: the rational person is not too sleepy, drunk, tired, confused, ...

Quote from Kreisel

Quote

The *sense* of a mathematical assertion denoted by a linguistic object A is intuitionistically determined (or understood) if we have laid down what constructions constitute a *proof* of A, i.e., if we have a construction r_A such that, for any construction c, $r_A(c) = 0$ if c is a proof of A and $r_A(c) = 1$ if c is not a proof of A: the logical particles in this explanation are interpreted truth functionally, since we are adopting the basic intuitionistic idealization that we can recognize a proof when we see one, and so r_A is decidable. (Note that this applies to *proof*, not *provability*.)

From: Georg Kreisel. *Foundations of intuitionistic logic*. In: E. Nagel, P. Suppes, A. Tarski (eds.), Logic, Methodology and Philosophy of Science, Proceedings of the 1960 International Congress, pp. 198–210. Stanford University Press, Stanford (1962).

Does type theory live up to this ideal?

Puzzle

Suppose A is the Ackermann function. One of the following is derivable in type theory.

$$\vdash \mathbf{refl}(A(3, 2^{65536} - 3)) \in \mathrm{Id}(\mathbb{N}, A(3, 2^{65536} - 3), A(4, 3)).$$

$$\vdash \mathbf{refl}(A(3, 2^{65535} - 3)) \in \mathrm{Id}(\mathbb{N}, A(3, 2^{65535} - 3), A(4, 3)).$$

Can you tell which one?

Does type theory live up to this ideal?

Puzzle

Suppose A is the Ackermann function. One of the following is derivable in type theory.

$$\vdash \mathbf{refl}(A(3, 2^{65536} - 3)) \in \mathrm{Id}(\mathbb{N}, A(3, 2^{65536} - 3), A(4, 3)).$$

$$\vdash \mathbf{refl}(A(3, 2^{65535} - 3)) \in \mathrm{Id}(\mathbb{N}, A(3, 2^{65535} - 3), A(4, 3)).$$

Can you tell which one?

Clearly, we also make the idealisation that the rational person has more time than the physical universe.

Is this really a proof?

Theorem

There are no positive integers a, b, c and n with $n \geq 3$ such that $a^n + b^n = c^n$.

Proof.

By a systematic search through all possible derivations with less than $2^{65536} - 3$ symbols. □

Theorem (?)

The non-trivial zeros of Riemann's $\zeta(s)$ function all lie on the complex line $\Re s = \frac{1}{2}$.

Proof.

By a systematic search through all possible derivations with less than $2^{65536} - 3$ symbols. □

What do we learn from this?

Proofs in type theory are rather different from proofs that you can follow "with your finger": you follow every line in the argument and then at the end you are convinced that the theorem is true.

To see that $a \in \sigma$ is true you may have to do *many* conversions, none of which are stored in the proof term.

A proof is also conventionally understood as providing an *explanation*: it explains why something is true. It is also debatable whether this applies to proof terms in type theory.

Is this just philosophy?

Theorem (Statman, 1979)

Equality in the typed lambda calculus is not elementary recursive.

Quote from Geuvers and Wiedijk

Quote

In theorem provers based on type theory the main performance bottleneck is the convertibility check: if the calculated type of a term M is A, but it used in a context where the type should be B, then the system needs to verify that $A =_{\beta\iota\delta} B$, where δ is the equality arising from definitional expansion (unfolding definitions) and ι is the equality arising from functions defined by (higher order primitive) recursion. In fact, the inefficiency of the convertibility check means that type correctness is in practice only semi-decidable. Although in theory it is decidable whether a term M has type A, in practice when it is not correct the system could be endlessly reducing and would not terminate in an acceptable time anymore.

From: Herman Geuvers and Freek Wiedijk. *A logical framework with explicit conversions*. Electronic Notes in Theoretical Computer Science 199 (2008), 33-47.

Explicit conversions

This motivates Geuvers & Wiedijk to write down a system with *explicit conversions*, where the conversions are stored in the proof term. For this they use heterogeneous ("John Major") equality.

Today: another solution. I will still use explicit conversions, but equality will be homogeneous.

Plan

Eliminate definitional equality completely and replace all the computation rules by propositional equalities, starting with the identity type.

Section 2

Homotopy type theory with explicit conversions

Basic rules

There will be only three judgement forms:

$$\vdash \Gamma \; ctxt \quad \Gamma \vdash \sigma \; type \quad \Gamma \vdash a \in \sigma$$

There will be only three basic rules, two of which are context formation rules:

$$\frac{}{\vdash [] \; ctxt} \qquad \frac{\vdash \Gamma \; ctxt \qquad \Gamma \vdash \sigma \; type}{\vdash [\Gamma, x \in \sigma] \; ctxt}$$

The only other rule is the following variable rule:

$$\frac{\vdash [\Gamma, x \in \sigma, \Delta] \; ctxt}{\Gamma, x \in \sigma, \Delta \vdash x \in \sigma}$$

Propositional identity types

$$\frac{\Gamma \vdash a \in \rho \qquad \Gamma \vdash b \in \rho}{\Gamma \vdash a =_\rho b \; type}$$

$$\frac{\Gamma \vdash a \in \rho}{\Gamma \vdash \mathbf{refl}(a) \in a =_\rho a}$$

$$\frac{\begin{array}{l}\Gamma, x \in \rho, y \in \rho, z \in x =_\rho y \vdash \sigma \; type \\ \Gamma \vdash p \in a =_\rho b \\ \Gamma, x \in \rho \vdash d \in \sigma[x, x, \mathbf{refl}(x)/x, y, z]\end{array}}{\Gamma \vdash \mathbf{idrec}(a, b, p, [x]d, [x, y, z]\sigma) \in \sigma[a, b, p/x, y, z]}$$

In addition, there is the term

$$\mathbf{idconv}(a, [x]d, [x, y, z]\sigma)$$

of type

$$\mathbf{idrec}(a, a, \mathbf{refl}(a), [x]d, [x, y, z]\sigma) =_{\sigma[a,a,\mathbf{refl}(a)/x,y,z]} d[a/x].$$

Propositional Π-types

$$\frac{\Gamma, x \in \sigma \vdash \tau \; \mathit{type}}{\Gamma \vdash \Pi x \in \sigma.\tau \; \mathit{type}}$$

$$\frac{\Gamma, x \in \sigma \vdash t \in \tau}{\Gamma \vdash \lambda x \in \sigma.t \; : \; \Pi x \in \sigma.\tau}$$

$$\frac{\Gamma \vdash f \in \Pi x \in \sigma.\tau \qquad \Gamma \vdash s \in \sigma}{\Gamma \vdash \mathbf{app}(f, s) \in \tau[s/x]}$$

$$\frac{\Gamma, x \in \sigma \vdash t \in \tau \qquad \Gamma \vdash s \in \sigma}{\Gamma \vdash \mathbf{betaconv}([x \in \sigma].t, s) \in \mathbf{app}(\lambda x \in \sigma.t, s) =_{\tau[s/x]} t[s/x]}$$

Propositional natural numbers type

$$\frac{\vdash \Gamma \; ctxt}{\Gamma \vdash \mathbb{N} \; type}$$

$$\frac{\vdash \Gamma \; ctxt}{\Gamma \vdash 0 \in \mathbb{N}} \qquad \frac{\Gamma \vdash t \in \mathbb{N}}{\Gamma \vdash \mathbf{succ}(t) \in \mathbb{N}}$$

$$\frac{\begin{array}{l}\Gamma, x \in \mathbb{N} \vdash \sigma \; type \\ \Gamma \vdash a \in \sigma[0/x] \\ \Gamma, x \in \mathbb{N}, y \in \sigma \vdash b \in \sigma[\mathbf{succ}(x)/x] \\ \Gamma \vdash t \in \mathbb{N}\end{array}}{\Gamma \vdash \mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, t) \in \sigma[t/x]}$$

Propositional natural numbers type, continued

Under the same assumptions as the elimination rule, there should be terms:

$$\mathbf{natconvzero}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b) \in$$
$$\mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, 0) =_{\sigma[0/x]} a$$

$$\mathbf{natconvsucc}([x \in \mathbb{N}]P, a, [x \in \mathbb{N}, y \in \sigma]b, t) \in$$
$$\mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, \mathbf{succ}(t)) =_{\sigma[\mathbf{succ}(t)/x]}$$
$$b[t, \mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, t)/x, y]$$

We could call the resulting system *propositional type theory* or *homotopy type theory with explicit conversions*.

Martin-Löf's proposal for a name

Propositional natural numbers type, continued

Under the same assumptions as the elimination rule, there should be terms:

$$\mathbf{natconvzero}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b) \in \mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, 0) =_{\sigma[0/x]} a$$

$$\mathbf{natconvsucc}([x \in \mathbb{N}]P, a, [x \in \mathbb{N}, y \in \sigma]b, t) \in \mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, \mathbf{succ}(t)) =_{\sigma[\mathbf{succ}(t)/x]} b[t, \mathbf{natrec}([x \in \mathbb{N}]\sigma, a, [x \in \mathbb{N}, y \in \sigma]b, t)/x, y]$$

We could call the resulting system *propositional type theory* or *homotopy type theory with explicit conversions*.

Martin-Löf's proposal for a name

Extensional type theory.

Basic results

Lemma

If $\Gamma \vdash a \in \sigma$ *and* $\Gamma \vdash a \in \tau$, *then* $\sigma \equiv \tau$ *(syntactic equality).*

Lemma

If $\Gamma \vdash a \in \sigma$ *is derivable, then it has precisely one derivation.*

Theorem

The question whether $\Gamma \vdash a \in \sigma$ *is derivable or not can be decided in linear time.*

Proof.

Straightforward backwards proof search. The last step of the derivation can be read off from a. □

In fact, in this system terms code *derivations*.

Section 3

Conjectures and questions

Conjectures and questions

Claim/conjecture

All the basic stuff of HoTT (first so many chapters of the HoTT book) can be formalised in HoTT with explicit conversions (extended with function extensionality, univalence, or ...).

See my papers on path categories (one of them with Ieke Moerdijk). Also: Coquand and Danielsson, "Isomorphism is equality".

Conjecture (homotopy canonicity)

From a derivation of $\vdash t \in \mathbb{N}$ one can effectively extract a natural number n, a term p and a derivation of $\vdash p \in t =_{\mathbb{N}} \mathbf{succ}^n(0)$.

Question

What is the precise relationship with traditional type theory? (Exercise: Formulate and prove a precise coherence/conservation theorem.)

Section 4

More motivation

Propositional type theory

Arguments in favour of propositional type theory (or homotopy type theory with explicit conversions):

- Type-checking efficiently decidable
- Proofs become explanatory
- Natural from homotopy-theoretic point of view (think: path types in cubical sets, path categories are a nice semantics)
- Leads to interesting metamathematical questions: does weak implies strong functionality still hold in propositional type theory? (Or different models of propositional resizing?)

Propositional type theory

Drawbacks of propositional type theory (or homotopy type theory with explicit conversions):

- Proof terms become very long!

Question

Does propositional type theory have any practical relevance?

A proof assistant would be more in the style of HOL (with explicit conversions) than conventional type theory. If one wanted to do this, one probably should look there how the HOL-people deal with very long proofs.

Understanding definitional equality

One other reason for pursuing such a project is in order to understand definitional equality. To best way to understand what something does for you is to try to live without it.

Even when you love definitional equality and you really want to have it, you are always faced with questions like:

- Which equalities should be definitional and which ones are not?
- I mean, it is sometimes rather arbitrary which equalities are propositional and which are definitional in standard type theory.
- Which one of $0 + x = x$ and $x + 0 = x$ is true definitionally and which one is not?
- Should it be decidable when two terms are definitionally equal and if so, how complex should we let the decision procedure become?
- Is there a rational basis for making such decisions?

A quote from Larry Paulson

Quote

My personal disenchantment with dependent type theories coincides with the decision to shift from extensional to intensional equality. This meant for example that $0 + n = n$ and $n + 0 = n$ would henceforth be regarded as fundamentally different assertions, one an identity holding by definition and the other a mere equality proved by induction. Of course I was personally upset to see several years of work, along with Constable's Nuprl project, suddenly put beyond the pale. But I also had the feeling that this decision had been imposed on the community rather than arising from rational discussion. And I see the entire homotopy type theory effort as an attempt to make equality reasonable again. (From: "Formalising mathematics in simple type theory". In: "Reflections on the Foundations of Mathematics", Springer, to appear.)

Equality for humans and machines

One could argue that the two forms of equality reflect a division of labour.

- Definitional equality: something the machine checks
- Propositional equality: something the human has to prove

But, actually, it is not clear. . .

- that the machine should normalise.
- normalisation is the only thing the machine should do.

This also does not seem to be what happens in reality.

More realistic picture: the default option is that the human does everything and then we let pragmatic considerations decide what to leave to the machine.

But for that it may be good to first to agree on what definitional equality is and what it should do for us. And understanding propositional type theory might be a good first step.

Thank you!